

Reviewer Pack — Mansour L1 Fourier Mass of Clause-Falsification Polynomial B...

Entry 9076d5e87956 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-18 05:56:31 UTC

Mansour L1 Fourier Mass of Clause-Falsification Polynomial Bounds DPLL Size

Entry ID: 9076d5e87956 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-18 05:56:31 UTC

1. Conjecture

Field A (mathematical branch): Fourier analysis of Boolean functions — specifically the Linial-Mansour-Nisan / Mansour L^1 Fourier mass $L_1(p) := \sum_S |\hat{p}(S)|$ of low-degree pseudo-Boolean polynomials in the Bonami-Beckner-Mansour tradition; distinct from the 4-norm/2-norm Bonami ratio (blacklisted) and from the cross-side symmetric-difference mass (blacklisted), since L_1 is the sum-of-absolute-coefficients functional underlying Mansour's DNF conjecture and the LMN approximation theorem. An arXiv search for 'Mansour L1 Fourier mass' AND ('Resolution refutation' OR 'DPLL' OR 'tree-Resolution') returns 0 direct hits and <5 adjacent papers, none using L_1 of the clause-count polynomial as a CNF-structural invariant. **Field B** (complexity object): Tree-like Resolution / DPLL refutation size $t(F)$ for unsatisfiable 3-CNFs F in the Ben-Sasson-Wigderson / Krajíček bounded-arithmetic regime, where $\neg F$ is Σ^b_0 and $\log_2 t(F)$ lower-bounds V^0 -witnessing complexity; structural shielding from Razborov-Rudich is provided because L_1 depends on the clause LIST (multiplicities, polarities), so two unsat CNFs with identical satisfiability functions (both $\equiv$ FALSE) can have very different L_1 .

Statement:

For an unsatisfiable 3-CNF $F = \{C_1, \dots, C_m\}$ on n Boolean variables, let $p_F: \{-1, +1\}^n \rightarrow \mathbb{Z}$, $p_F(x) := \#\{j : x \text{ falsifies } C_j\}$ be the clause-falsification count polynomial, expand it in the Walsh basis $p_F(x) = \sum_S \hat{p}_F(S) \chi_S(x)$, and define the structural Fourier L^1 mass $L_1(F) := \sum_{1 \leq |S| \leq 3} |\hat{p}_F(S)|$. We conjecture that for every unsatisfiable 3-CNF F there is an absolute constant $c \in [0.25, 3.0]$ such that $L_1(F) / \sqrt{m+1} \leq c \cdot \sqrt{(\log_2(t(F) + 1) + 1)}$, where $t(F)$ is the DPLL tree size under the most-frequent-variable branching heuristic; a single unsat F violating the inequality refutes the conjecture.

Rationale (proposer's reasoning):

Mansour's conjecture ($\text{DNF } L_1 \leq s^{O(\log \log s)}$) and the LMN low-degree concentration theorem tie Boolean-function L^1 Fourier mass to small-formula computability; transposing this to CNF refutations predicts that 'Fourier-spread' clause sets (large L_1 per $\sqrt{m}$) are forced to have correspondingly deep search trees, since each Walsh monomial in p_F encodes a clause-polarity correlation that any DPLL path must locally certify. The Tseitin family is a sharp test bed: every $\text{XOR}=\omega(v)$ constraint contributes Fourier mass concentrated exactly at one size-3 monomial of magnitude $1/2$ (lower levels cancel by parity), giving $L_1(\text{Tseitin})/\sqrt{m} = \Theta(\sqrt{V})$, parallel to the $\Omega(V)$ Ben-Sasson-Wigderson width

lower bound. The invariant is structural (clause-list-dependent), not algebrizing (uses $|\cdot|$ not ring extensions), and not relativizing.

Taxonomy category: FOURIER_ANALYTIC (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 55863d38ffc967f1

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 30 seeds and all three instance families (Tseitin, random unsat 3-CNF at density 5, trivial-unsat) with $n \leq 36$, every generated unsatisfiable F must satisfy $L_1(F)/\sqrt{m+1} \leq 3 \cdot \sqrt{\log_2(t^*(F)+1)+1}$; a single violating instance falsifies.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.93	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.90	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Mansour L1 Fourier mass CNF Resolution refutation - Fourier analysis Boolean functions DPLL tree size lower bound - Walsh spectrum clause falsification polynomial tree-Resolution

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=0.0s

5.1 Generated Python source

```
import sys
import random
import math
import itertools
from collections import defaultdict

def generate_random_3cnf(n, m, seed):
    random.seed(seed)
    variables = list(range(1, n + 1))
    clauses = []
    for _ in range(m):
        clause = random.sample(variables, 3)
        for i in range(3):
```

```

        if random.random() < 0.5:
            clause[i] = -clause[i]
        clauses.append(clause)
    return clauses

def generate_tseitin_cnf(n, seed):
    random.seed(seed)
    variables = list(range(1, n + 1))
    edges = list(itertools.combinations(variables, 2))
    random.shuffle(edges)
    clauses = []
    for u, v in edges[:n]:
        w = random.choice(variables)
        clauses.append([u, v, -w])
        clauses.append([u, -v, w])
        clauses.append([-u, v, w])
        clauses.append([-u, -v, -w])
    return clauses

def generate_trivial_unsat(n, m, seed):
    random.seed(seed)
    variables = list(range(1, n + 1))
    clauses = []
    for _ in range(m):
        clause = random.sample(variables, 3)
        for i in range(3):
            clause[i] = -clause[i]
        clauses.append(clause)
    return clauses

def is_satisfiable(clauses, assignment):
    for clause in clauses:
        if all((lit > 0 and assignment.get(abs(lit), False)) or (lit < 0 and not assignment.get(abs(lit), False)) for lit in clause):
            return True
    return False

def dpll(clauses, assignment, variables, max_nodes):
    if max_nodes <= 0:
        return None
    if not clauses:
        return assignment
    if any(not clause for clause in clauses):
        return None
    var = max(variables, key=lambda v: sum(1 for clause in clauses if v in clause or -v in clause))
    for value in [True, False]:
        new_assignment = assignment.copy()
        new_assignment[var] = value
        new_clauses = []
        for clause in clauses:
            new_clause = [lit for lit in clause if not ((lit > 0 and new_assignment.get(lit, False)) or (lit < 0 and not new_assignment.get(abs(lit), False)))]
            if not new_clause:
                return None
            new_clauses.append(new_clause)
        result = dpll(new_clauses, new_assignment, [v for v in variables if v != var], max_nodes - 1)
        if result is not None:
            return result

```

```

    return None

def compute_t_star(clauses, variables):
    max_nodes = 2 * 10**6
    result = dpll(clauses, {}, variables, max_nodes)
    if result is None:
        return max_nodes
    return max_nodes - sum(1 for _ in range(max_nodes) if dpll(clauses, {}, variables, _) is not None)

def compute_ll(clauses, n):
    ll = 0.0
    for clause in clauses:
        for S in itertools.product([-1, 1], repeat=n):
            if len([s for s in S if s == 1]) > 3:
                continue
            p = 1.0
            for lit in clause:
                if lit > 0:
                    p *= (1 + S[lit - 1]) / 2
                else:
                    p *= (1 - S[abs(lit) - 1]) / 2
            ll += abs(p)
    return ll

def run_trial(seed):
    random.seed(seed)
    n_values = [8, 10, 12, 14, 16, 18, 20]
    m_values = [5, 10, 15, 20, 25]
    instances_tested = 0
    metric_values = []
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        for m in m_values:
            if n * m > 36:
                continue
            for family in ['tseitin', 'random', 'trivial']:
                if family == 'tseitin':
                    clauses = generate_tseitin_cnf(n, seed)
                elif family == 'random':
                    clauses = generate_random_3cnf(n, m, seed)
                else:
                    clauses = generate_trivial_unsat(n, m, seed)
                if not is_satisfiable(clauses, {}):
                    ll = compute_ll(clauses, n)
                    t_star = compute_t_star(clauses, list(range(1, n + 1)))
                    metric_value = ll / math.sqrt(m + 1)
                    bound = 3 * math.sqrt(math.log2(t_star + 1) + 1)
                    if metric_value > bound:
                        conjecture_holds = False
                        counterexample = f"n={n}, m={m}, family={family}, ll={ll}, t_star={t_star}"
                        break
                    metric_values.append(metric_value)
                    instances_tested += 1
            if not conjecture_holds:

```

```

        break
    if not conjecture_holds:
        break

if not metric_values:
    return {
        "metric_name": "L1/Fourier_mass_ratio",
        "metric_value": 0.0,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "No valid instances generated"
    }

return {
    "metric_name": "L1/Fourier_mass_ratio",
    "metric_value": sum(metric_values) / len(metric_values),
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    seeds = sys.argv[1:] if len(sys.argv) > 1 else [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41]
    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    metric_values = [r["metric_value"] for r in results if r["metric_value"] != 0.0]
    if not metric_values:
        print("RESULT: INCONCLUSIVE reason=no_valid_instances")
        sys.exit(0)

    mean = sum(metric_values) / len(metric_values)
    std = math.sqrt(sum((x - mean) ** 2 for x in metric_values) / len(metric_values))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        counterexample = next(r["counterexample"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

nces_tested': 0, 'conjecture_holds': False, 'counterexample': 'No valid instances generated'}
TRIAL: {'metric_name': 'L1/Fourier_mass_ratio', 'metric_value': 0.0, 'instances_tested': 0, 'conjecture_holds': False, 'counterexample': 'No valid instances generated'}
TRIAL: {'metric_name': 'L1/Fourier_mass_ratio', 'metric_value': 0.0, 'instances_tested': 0, 'conjecture_holds': False, 'counterexample': 'No valid instances generated'}

```

```

TRIAL: {'metric_name': 'L1/Fourier_mass_ratio', 'metric_value': 0.0, 'instances_tested': 0, 'conjecture': 'L1/Fourier_mass_ratio'}
TRIAL: {'metric_name': 'L1/Fourier_mass_ratio', 'metric_value': 0.0, 'instances_tested': 0, 'conjecture': 'L1/Fourier_mass_ratio'}
TRIAL: {'metric_name': 'L1/Fourier_mass_ratio', 'metric_value': 0.0, 'instances_tested': 0, 'conjecture': 'L1/Fourier_mass_ratio'}
TRIAL: {'metric_name': 'L1/Fourier_mass_ratio', 'metric_value': 0.0, 'instances_tested': 0, 'conjecture': 'L1/Fourier_mass_ratio'}
TRIAL: {'metric_name': 'L1/Fourier_mass_ratio', 'metric_value': 0.0, 'instances_tested': 0, 'conjecture': 'L1/Fourier_mass_ratio'}
TRIAL: {'metric_name': 'L1/Fourier_mass_ratio', 'metric_value': 0.0, 'instances_tested': 0, 'conjecture': 'L1/Fourier_mass_ratio'}
TRIAL: {'metric_name': 'L1/Fourier_mass_ratio', 'metric_value': 0.0, 'instances_tested': 0, 'conjecture': 'L1/Fourier_mass_ratio'}
TRIAL: {'metric_name': 'L1/Fourier_mass_ratio', 'metric_value': 0.0, 'instances_tested': 0, 'conjecture': 'L1/Fourier_mass_ratio'}
TRIAL: {'metric_name': 'L1/Fourier_mass_ratio', 'metric_value': 0.0, 'instances_tested': 0, 'conjecture': 'L1/Fourier_mass_ratio'}
TRIAL: {'metric_name': 'L1/Fourier_mass_ratio', 'metric_value': 0.0, 'instances_tested': 0, 'conjecture': 'L1/Fourier_mass_ratio'}
RESULT: INCONCLUSIVE reason=no_valid_instances

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

The stdout tail shows ‘instances_tested’: 0 across every trial and a final ‘RESULT: INCONCLUSIVE reason=no_valid_instances’ — no unsat 3-CNF was actually generated or evaluated, so the conjecture has zero empirical support. This is a construction gap / metric definition bug: the harness never computed $L_1(F)$ or $t^*(F)$ on any instance, making any SUPPORTED verdict vacuous. Additionally, the metric_value of 0.0 trivially satisfies the inequality, which would falsely register as ‘holds’ if the aggregate

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test generated zero valid instances across all trials (RESULT: INCONCLUSIVE reason=no_valid_instances), so the pre-registered support condition of ≥ 200 instances was not met and no empirical evaluation of $L_1(F)$ or $t^*(F)$ actually occurred. | next: Fix the instance generator to reliably produce unsatisfiable 3-CNFs from the Tseitin, random-density-5, and trivial-unsat families, verify nonzero instances_tested, and re-run with the 30-seed protocol.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	487464
2	preregistration	claude_max	opus	0	0	8530
3	novelty	claude_max	opus	0	0	3503
4	novelty	claude_max	opus	0	0	7126
5	test_gen	mistral	codestral-latest	0	0	319116
6	test_gen	mistral	codestral-latest	0	0	313252
7	test_gen	mistral	codestral-latest	0	0	318959
8	test_gen	mistral	codestral-latest	0	0	312039
9	critic	claude_max	opus	0	0	9718
10	judge	claude_max	opus	0	0	5269

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 1784976 ms total latency. Provider mix: {‘claude_max’: 6, ‘mistral’: 4}

(full prompt+response transcripts available in [research/audit/9076d5e87956.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9076d5e87956.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/9076d5e87956.tar.gz` (if generated)